

Course ID: SW 302

User Interface (UI) Development

Dr. Mohamed Sami Rakha

Lecture#3

Introduction to CSS

Lecture Outline

- What is CSS?
- Why do we need CSS?

Dr. Mohamed Sami Rakha

What is CSS?

What is CSS?

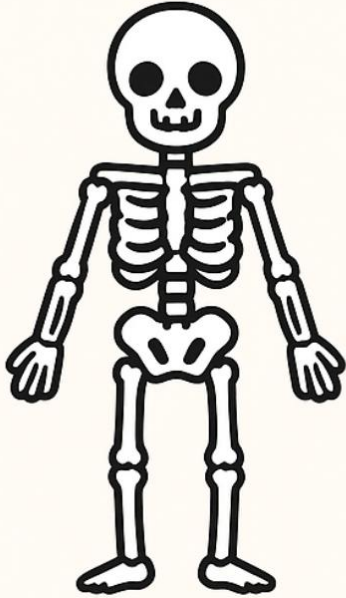
Cascading Style Sheets (CSS) were established by the World Wide Web Consortium (W3C). The CSS specification allows for more control over the look, or style, of web pages by providing a central location, or sheet, where you can define how certain HTML pages are going to be interpreted by the browser.

Why is it called “cascading”?

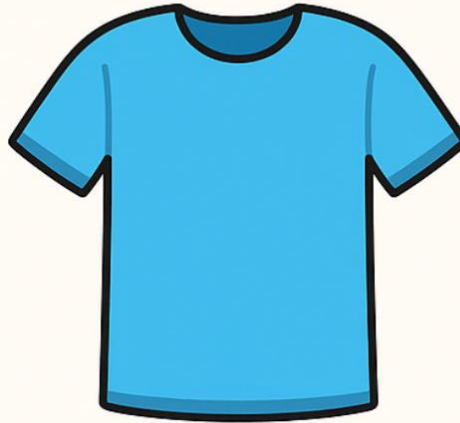
In CSS, multiple styles can be applied to a particular web page. The browser will interpret these styles in a **cascading fashion... ?**

HTML vs CSS vs JavaScript

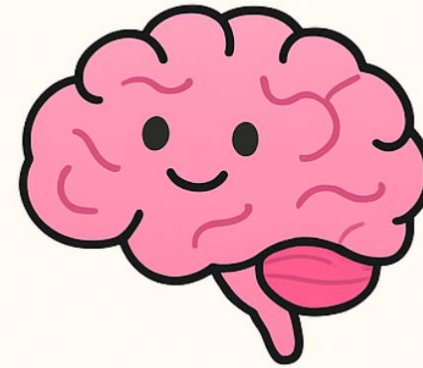
Dr. Mohamed Sami Rakha



HTML
Skeleton
(structure)



CSS
Clothes
(style)



JavaScript
Muscles/Brain
(behavior & interactivity)

Style Separate from Structure

These pages have the **exact same HTML** source but different style sheets:



Why do we need CSS?

Why do we need CSS?

- **CSS saves time:** When most of us first learn **HTML**, we get taught to set the **font face, size, colour, style**, etc, every time it occurs on a page. This means we find ourselves typing (or copying & pasting) the same thing over and over again. With CSS, you only have to specify these details once for any element. CSS will automatically apply the specified styles whenever that element occurs.
- **Pages load faster:** Less code means faster download times.
- **Easy maintenance:** To change the style of an element, you only have to make an edit in one place.
- **Superior styles to HTML:** CSS has a much wider array of attributes than HTML.
- **Greater designer control of the appearance of the page.**

How Style Sheets Work

1. Start with a marked-up document (like HTML).
2. Write styles for how you want elements to look using CSS syntax.
3. Attach the styles to the document (**there are a number of ways**).
4. The browser uses your instructions when rendering the elements.

Attach the Styles: CSS Definition Locations

1- Inline styles: the “style” attribute

Dr. Mohamed Sami Rakha

```
<p style="font-color:red;font-size:10px;">Content</p>
```

Note: the selector for inline CSS is the tag that contains the style attribute.

2- Embedded style sheets: the <style> markup tag

```
<html><head><style>
p {      background-color: Red;
          font-family: serif;
          font-color: White; }
</style></head><body>
<p>Content</p>
</body></html>
```

3- External style sheets: the .css stylesheet file

```
<link rel="stylesheet" type="text/css" href="mystylesheet.css" />
```

Inline Styles

Apply a style declaration to a single element with the Style attribute

```
<p style="font-size: large;">Paragraph text|...</p>
```

To add multiple properties, separate them with semicolons:

```
<h3 style="color: red; margin-top: 30px;">Intro</h3>
```

Embedded Style Sheets

Embedded style sheets are placed in the head of the document via the style element:

```
<head>
  <title>Titles are required</title>
  <style>
    /* style rules go here */
  </style>
</head>
```

External Style Sheets

The style rules are saved in a separate text only .css file and attached via link or @import

Via **link** element in HTML:

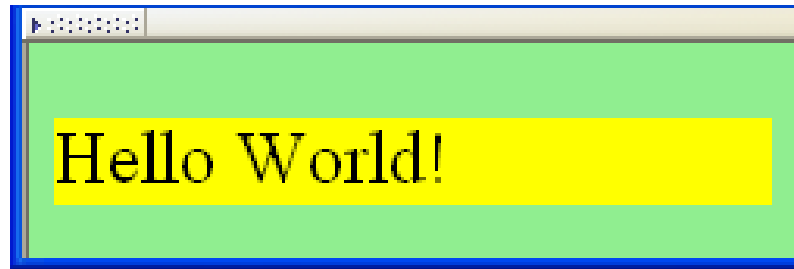
```
<head>
  <title>Titles are require</title>
  <link rel="stylesheet" href="/path/example.css">
</head>
```

Via **@import** rule in a style sheet:

```
<head>
  <title>Titles are required</title>
  <style>
    @import url("/path/example.css");
    p {font-face: Verdana;}
  </style>
</head>
```

A styled HTML document

Each rule selects an **HTML** element and specifies how it should be displayed.



Produced by the style sheet **style1.css**:

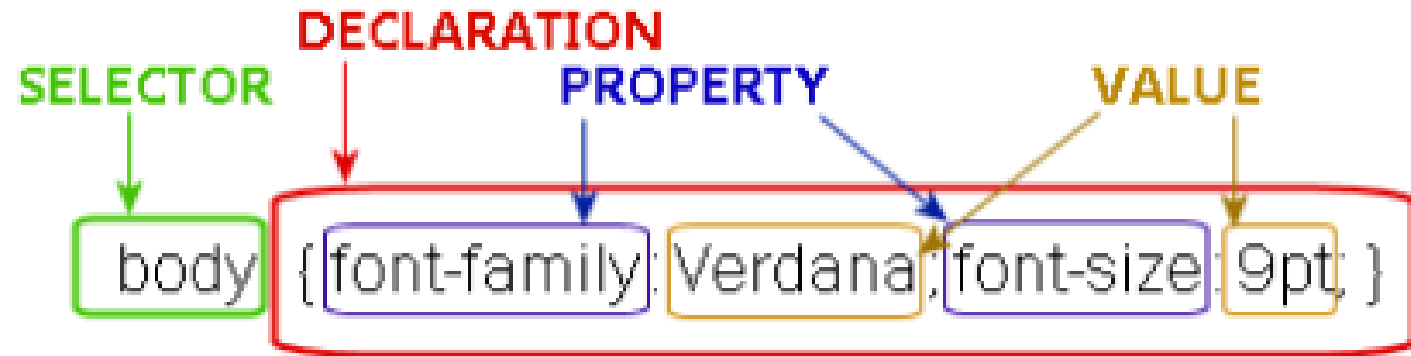
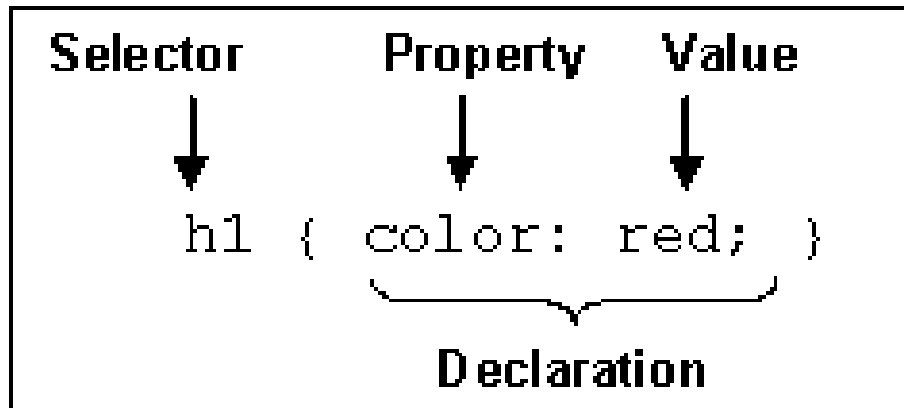
```
body { background-color:lime }  
p    { font-size:x-large; background-color:yellow }
```

CSS Syntax: Style Rule Structure

Four Elements to a CSS Statement:

Dr. Mohamed Sami Rakha

- **Selector:** What HTML elements/ sections does it affect?
- **Declaration:** the style that should be applied to the selected element.
- **Property:** What attribute of that HTML section will be affected?
- **Value:** What change will be made to that attribute?



CSS Syntax

Dr. Mohamed Sami Rakha

Map the elements in the example below?

```
p {  
  color: blue;  
}
```

- **Selector:** ?
- **Declaration:** ?
- **Property:** ?
- **Value:** ?

CSS Selector Types

A selector in CSS is the pattern used to target HTML elements / sections that you want to style.

It tells the browser:

“Which element(s) / sections in the HTML should this style apply to?”

There are many kinds of selectors and many ways to reference them:

1-Tag

2- ID

3- Class

Tag Selector

HTML Type Tag – selected with the tag type

- This selector selects tags by tag or “element” name.
- This selector must match one or more HTML tag elements of the same name.
- **Easiest to use.**

```
p { font-size: 10px;  
    font-color: White; }
```

```
<p>Content</p>
```

ID Selector

- The ID selector as a unique identifier to an HTML element.
- The id selector is used if you want to select a single, unique element.
- CSS uses a hash or pound sign (#) to indicate it's ID
- ID's are unique
- IDs in theory, are only supposed to be used once per page
- In other words, a page would not be validate by W3C standards.
- Anytime we see ID in the computer world, think “unique”.

ID Selector

Dr. Mohamed Sami Rakha

```
#text1 {  
  text-align: center;  
  color: green;  
}  
<h1 id="text1">Think Spring!</h1>
```

Think Spring!

Class Selector

- This selector allows you to style content irrespective of what tag you apply it to
- Classes are **NOT unique**
- Use the same class **on multiple element types.**
- Use multiple classes on the same element.

Class Selector

Dr. Mohamed Sami Rakha

```
.center {  
    text-align: center;  
    color: green;  
}  
  
<h1 class="center">Think Spring!</h1>  
<p class="center">Think Spring!</p>
```

Think Spring!

Think Spring!

ID vs Class selectors

ID Selector

- **ID (#idName):**
- Must be unique on a page.
- Used to identify one **specific** element.
- Has higher specificity than class.

Dr. Mohamed Sami Rakha

- **Class (.className):**
- Can be applied to **multiple elements**.
- Used for **groups** of elements that share the same style.

Best Practice: Use **classes** for styling, use **IDs** for JavaScript hooks, or when a single element must be uniquely styled.

What will happen if I give the same ID to multiple elements on an HTML page???

Problems cause of multiple elements with same ID

- If you give the same ID to multiple elements: **The styles will be applied to all elements with that ID.** But could cause problems like the following:
- **JavaScript may break:** `document.getElementById("note")` will only return the first element with that ID, ignoring the rest. This leads to confusion and bugs.
- **Accessibility & SEO issues:** IDs are used by screen readers, ARIA (Accessible Rich Internet Applications) labels, and anchors. Duplicate IDs can confuse assistive tech and harm accessibility.

Best Practice: If you need to style multiple elements the same way, use a **class** selector instead!..

So... Why is it called “cascading”?

So... Why is it called “cascading”?

In CSS, multiple styles can be applied to a particular web page. The browser will interpret these styles in a **cascading fashion**...

- Style rules set up site-wide are **overridden** by styles located within individual pages.
- Individual pages are **overridden** by styles inside an individual tag.
- In addition, the end user can set up styles in the browser that will override the author's styles.
- All matching rules for a particular **selector** will be applied, except where they conflict with each other (in which case, the latter rule would be applied, as determined by the **cascade weight**).

So... Why is it called “cascading”?

The cascade refers to the system for **resolving conflicts when several styles apply to the same element**.

Style information is passed down (it “cascades” down) until overwritten by a style rule with more **weight**.

Cascade Weight is considered based on:

- **Priority** of style rule source
- **Specificity** of the selector
- **Rule order**

The Cascade: Style Weight

Dr. Mohamed Sami Rakha

1- Importance: Styles marked **!important** override everything else (but should be avoided unless absolutely necessary).

```
p { color: blue !important; } /* wins */  
p { color: red; }
```

The Cascade: Style Weight

2. Specificity (how *precisely* a rule targets the element)

- Inline style
- ID selectors (#id).
- Class selectors (.class, [attr]).
- Element selectors (div, p, h1).

Specificity weight (simplified):

Inline styles = **1000**

ID = **100**

Class/attribute/pseudo-class = **10**

Element/pseudo-element = **1**

The Cascade: Style Weight

```
p { color: blue; }      /* specificity 1 */  
.intro { color: green; } /* specificity 10 */  
#main { color: red; }   /* specificity 100 */
```

If the code

```
<p id="main" class="intro">Text</p>
```

What is the colour of the “Text”?

It will be **red**?

The Cascade: Style Weight

Specificity Value Order



N/A (highest)

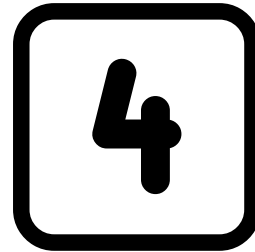
1000

100

10

1

!important > inline > ID > class > element > **later rule wins**



The Cascade: Style Weight

3. Source Order: When two rules have equal specificity, rule order is used.

Whichever rule appears last wins.

```
<style>
  p {color: red;}
  p {color: blue;}
  p {color: green;}
</style>
```

In this example, paragraphs would be green.

- Styles may come in from external style sheets, embedded style rules, and inline styles. The style rule that gets parsed last (the **one closest to the content**) will apply.

The Cascade: Style Weight: Example

```
<p id="note" class="highlight">Hello World</p>
```

```
p { color: black; }  
.highlight { color: green; }  
#note { color: red; }  
p#note.highlight { color: blue; }  
p { color: purple !important; }
```

What would be the colour of the text “Hello World”?

The Cascade: Style Weight: Example

```
<p id="note" class="highlight">Hello World</p>
```

```
p { color: black; }  
.highlight { color: green; }  
#note { color: red; }  
p#note.highlight { color: blue; }  
p { color: purple !important; }
```

What would be the colour of the text “Hello World”?

Result → The text will be purple (because of !important).

The Cascade: Style Weight: Example

```
<p id="note" class="highlight">Hello World</p>
```

```
p { color: black; }  
.highlight { color: green; }  
#note { color: red; }  
p { color: black; }  
#note { color: blue; }
```

Now, What would be the colour of the text “Hello World”?

The Cascade: Style Weight: Example

```
<p id="note" class="highlight">Hello World</p>
```

```
p { color: black; }  
.highlight { color: green; }  
#note { color: red; }  
p { color: black; }  
#note { color: blue; }
```

Now, What would be the colour of the text “Hello World”?

The `<p>` will be **blue**.

The Cascade: Style Weight: Example

```
<p id="note" class="highlight">Hello World</p>
```

```
p { color: black; }  
.highlight { color: green; }  
p#note.highlight { color: orange; }  
#note { color: red; }
```

Now, What would be the colour of the text “Hello World”?

The Cascade: Style Weight: Example

```
<p id="note" class="highlight">Hello World</p>
```

p { color: black; } → (Specificity =1)
.highlight { color: green; } → (Specificity =10)
p#note.highlight { color: orange; } → (Specificity =1 (for element)
+ 100 (for ID) + 10 (for class) = 111)
#note { color: red; } → (Specificity =100)

Now, What would be the colour of the text “Hello World”?

The <p> will be **orange**.

Dr. Mohamed Sami Rakha

Thank You...